Universidad Nacional Autó
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 2.9

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 21/09/2026

1. Base de Conocimiento (Prolog)

La base de conocimiento define la ubicación ideal de los productos y las reglas para el diagnóstico y la planificación

2. Estados Iniciales y Finales

Para un programa de toma de decisiones, definimos los estados de la siguiente manera:Estado Inicial: El robot se encuentra en la "posición inicial" (centro del triángulo que forman los estantes). Los estantes tienen una distribución de productos que puede o no coincidir con la ideal.Estado Final:

- 1. El cliente ha recibido el producto solicitado (ej. "Aquí está el refresco").

- 2. Todos los productos observados durante la tarea han sido reubicados en sus estantes correctos según la base de conocimiento ("Productos Acomodados").

3. Integración en Python (Uso de pyswip)

Para conectar esto con Python, se utiliza la librería pyswip. El flujo consiste en que Python maneja la interfaz y el robot, mientras Prolog toma las decisiones lógicas.

```
# 1. Instalar el motor de SWI-Prolog en el sistema
!apt-get install -y swi-prolog

# 2. Instalar la librería puente para Python
!pip install pyswip
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
swi-prolog is already the newest version (9.0.4+dfsg-3.1ubuntu4).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Requirement already satisfied: pyswip in /usr/local/lib/python3.13/dist-packages
```

Crear un archivo como base de conocimiento

% Ubicaciones ideales (Creencia inicial del sistema)

ideal(estante_bebidas, [cerveza, refresco]).

ideal(estante_comida, [sopa, cereal]).

ideal(estante_pan, [galletas]).

% Estado actual (Hechos dinámicos que Python actualizará tras la observación)

% Ejemplo de un estado con desorden (Caso 1.2 b)

ubicado_en(estante_bebidas, refresco).

ubicado_en(estante_bebidas, cerveza).

ubicado_en(estante_bebidas, sopa). % Producto desacomodado

ubicado_en(estante_comida, cereal).

ubicado_en(estante_pan, galletas).

% Regla de Diagnóstico: Identifica qué productos no están en su lugar

diagnostico(Producto, EstanteActual, EstanteCorrecto) :-

```
ubicado_en(EstanteActual, Producto),

ideal(EstanteCorrecto, ProductosIdeales),

member(Producto, ProductosIdeales),

EstanteActual \= EstanteCorrecto.
```

% Regla de Planificación: Genera las acciones necesarias

plan(entregar(P)) :- ubicado_en(_, P).

plan(acomodar(P, Destino)) :- diagnostico(P, _, Destino).

```
%%writefile supermercado.pl
% Ubicaciones ideales (Creencia inicial del sistema)
ideal(estante_bebidas, [cerveza, refresco]).
ideal(estante_comida, [sopa, cereal]).
ideal(estante_pan, [galletas]).

% Estado actual (Ejemplo con desorden)
ubicado_en(estante_bebidas, refresco).
ubicado_en(estante_bebidas, cerveza).
ubicado_en(estante_bebidas, sopa). % Producto desacomodado
ubicado_en(estante_comida, cereal).
ubicado_en(estante_pan, galletas).

% Regla de Diagnóstico: Identifica qué productos no están en su lugar
diagnostico(Producto, EstanteActual, EstanteCorrecto) :-
    ubicado_en(EstanteActual, Producto),
    ideal(EstanteCorrecto, ProductosIdeales),
    member(Producto, ProductosIdeales),
    EstanteActual \= EstanteCorrecto.

% Regla de Planificación: Genera las acciones necesarias
plan(entregar(P)) :- ubicado_en(_, P).
plan(acomodar(P, Destino)) :- diagnostico(P, _, Destino).
```

```
Overwriting supermercado.pl
```

```
from pyswip import Prolog

prolog = Prolog()
prolog.consult("supermercado.pl") # Carga la base de conocimiento

def ejecutar_asistente(producto_buscado):
    print(f"Buscando: {producto_buscado}")
```

```
        # 1. Consultar a Prolog por el plan de entrega
        plan_entrega = list(prolog.query(f"plan(entregar({producto_buscado}))"))

        # 2. Consultar si hay algo que reacomodar (Diagnóstico)
        objetos_desacomodados = list(prolog.query("plan(acomodar(P, Destino))"))

        if objetos_desacomodados:
            for obj in objetos_desacomodados:
                print(f"Acción: Mover {obj['P']} al {obj['Destino']}")

        if plan_entrega:
            print(f"Acción: Entregar {producto_buscado} al cliente.")
        else:
            print("Error: Producto no encontrado en el sistema.")

    ejecutar_asistente("refresco")
```

```
Buscando: refresco
Acción: Mover sopa al estante_comida
Acción: Entregar refresco al cliente.
```

Escenario del Desafío:

El cliente solicita un refresco. El robot llega al "Estante de Bebidas" y lo encuentra vacío. Al inspeccionar el "Estante de Comida", descubre que allí están la cerveza, la sopa y las galletas, mientras que el refresco y el cereal están en el "Estante de Pan". **Reto para el programador:** Implementar una regla en Prolog que permita al robot priorizar: ¿Debe acomodar todo primero o entregar el refresco cuanto antes para satisfacer al cliente?.

Manejar el "Error de Búsqueda": Si el robot va a un estante esperando algo y no lo encuentra, debe disparar un nuevo diagnóstico (re-planificación en tiempo real).

Restricción: El robot solo puede cargar un máximo de dos objetos a la vez. ¿Cómo optimiza el movimiento entre los 3 estantes para minimizar la distancia recorrida?

Conclusiones finales del estudiante

En esta práctica vimos cómo conectar Python con Prolog usando la librería `pyswip` para dividir de forma ordenada las tareas de un robot: Python se encarga de la interfaz y la ejecución, mientras que Prolog razona sobre la base de conocimiento y decide qué pasos seguir

---

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados